

Flows

Contents

1 Homework solutions	1
1.a Longest common subword	1
1.b Held-Karp algorithm for the TSP	2
2 Flow vocabulary	2
2.a Flows from a source to a target	2
2.b Cuts between a source and a target	2
2.c Flows with arbitrary inputs and outputs	3
2.d Modeling examples	3
3 Flow algorithms	3
3.a Optimality criterion for maximum flows	3
3.a.a Weak duality	3
3.a.b Residual graph & augmenting paths	4
3.a.c Strong duality	4
3.b Ford-Fulkerson	5
3.b.a Pseudocode	5
3.b.b Complexity	5
3.c Edmonds-Karp	5
3.c.a Pseudocode	5
3.c.b Complexity	5
3.d Minimum mean cycle-canceling (for minimum cost b -flows)	6
4 Linear programming for flows	6
4.a Formulation	6
4.b Polyhedral interpretation	7
4.c Homework	7

1 Homework solutions

1.a Longest common subword

Exercise 5.19

As usual in dynamic programming, we generalize the problem. Let us try to compute the length $\ell(i, j)$ of the longest common subword of $w_1[1 : i]$ and $w_2[1 : j]$. There are several cases to consider:

1. The end letters of both words are identical:

$$\ell(n_1, n_2) = 1 + \ell(n_1 - 1, n_2 - 1) \quad (1)$$

2. The end letters of both words are different:

$$\ell(n_1, n_2) = \max\{\ell(n_1 - 1, n_2), \ell(n_1, n_2 - 1)\} \quad (2)$$

That way, we get the recursive Bellman equation. The initialization is done with

$$\ell(n_1, 0) = \ell(0, n_2) = 0 \quad (3)$$

1.b Held-Karp algorithm for the TSP

Exercise 5.20

1. We fix a source vertex $s \in V$ and use (X, v) as the state, with $X \subset V \setminus \{s\}$ and $v \in V \setminus X$. Let $\ell(X, v)$ be the length of the shortest tour starting at s , going through all the vertices of X and ending at v . We initialize $\ell(\emptyset, v) = c(s, v)$ and write the Bellman equation:

$$\ell(X, v) = \min\{\ell(X \setminus \{v\}, u) + c(u, v) : u \in X \setminus \{v\}\} \quad (4)$$

2. A naive algorithm would evaluate $n!$ solutions. We can do slightly better by fixing a starting vertex (divides complexity by n) and ignoring reflected tours (divides complexity by 2). The final enumeration still costs $\frac{(n-1)!}{2}$.

2 Flow vocabulary

2.a Flows from a source to a target

We consider a digraph $D = (V, A)$ with additional features:

- nonnegative upper capacities $u(a) \geq 0$ on each arc
- two special nodes: a source s and a target t

An s - t flow under u is a vector $f \in \mathbb{R}^A$ satisfying Kirchhoff's current law

$$\forall v \in V \setminus \{s, t\}, \quad \sum_{a \in \delta^-(v)} f(a) = \sum_{a \in \delta^+(v)} f(a) \quad (5)$$

and capacity constraints

$$\forall a \in A, \quad 0 \leq f(a) \leq u(a) \quad (6)$$

The value of an s - t flow f is the total quantity flowing out of the source:

$$\text{val}(f) = \sum_{a \in \delta^+(s)} f(a) - \sum_{a \in \delta^-(s)} f(a) \quad (7)$$

Maximum flow problem: Find an s - t flow under u of maximum value $\text{val}(f)$.

2.b Cuts between a source and a target

An s - t cut (S, T) is a partition $V = S \cup T$ of the vertices into disjoint subsets ($S \cap T = \emptyset$) such that $s \in S$ and $t \in T$. It can be viewed as a set of arcs $B = \delta^+(S)$ intersecting any s - t path.

The capacity of a cut is the sum of the capacities of its arcs:

$$u(S, T) = \sum_{\substack{i \in S \\ j \in T \\ (i, j) \in A}} u(i, j) \quad \text{or} \quad u(B) = \sum_{a \in B} u(a) \quad (8)$$

Minimum cut problem: Find an s - t cut of minimum capacity $u(B)$

2.c Flows with arbitrary inputs and outputs

For this setting we consider a digraph $D = (V, A)$ with slightly different features:

- lower and upper capacities $0 \leq \ell(a) \leq u(a)$ on each arc
- cost values $c(a) \geq 0$ on each arc
- algebraic inflows $b(v)$ at each vertex such that $\sum_{v \in V} b(v) = 0$

A b -flow under (ℓ, u) is a vector $f \in \mathbb{R}^A$ satisfying Kirchhoff's current law

$$\forall v \in V, \quad b(v) + \sum_{a \in \delta^-(v)} f(a) = \sum_{a \in \delta^+(v)} f(a) \quad (9)$$

and capacity constraints

$$\forall a \in A, \quad \ell(a) \leq f(a) \leq u(a) \quad (10)$$

If $b(v) = 0$ everywhere, we call f a circulation.

The cost of a b -flow f is the sum of the costs induced by f on each arc:

$$c(f) = \sum_{a \in A} c(a)f(a) \quad (11)$$

Minimum cost flow problem: find a b -flow under (ℓ, u) of minimum cost.

2.d Modeling examples

Exercise 6.8: Monge's transportation problem. We have n sand heaps of volume s_i each, and m holes to fill of volume t_j each. The moving cost per cubic meter between heap i and hole j is d_{ij} . How do we transport the mass while minimizing total cost?

The trick is to add a fictitious source s and target t . Also pay attention to the case where there is excess volume on either end. ■

Exercise 6.9: bus trip. A bus with capacity B leaves city 1 and visits cities $2, \dots, n$ in order. For each pair $i < j$ of cities, d_{ij} passengers want to make the trip, for a ticket price p_{ij} . How many passengers should the driver take in each city to maximize profit?

3 Flow algorithms

3.a Optimality criterion for maximum flows

Historical context: cold war.

3.a.a Weak duality

Proposition (Cuts as upper bounds on flows): Let $0 \leq f \leq u$ be an s - t flow and (S, T) be an s - t cut. Then

- the value of the flow is equal to the "net flow $f(S, T)$ across the cut ($S \rightarrow T$ minus $T \rightarrow S$)

- it is upper-bounded by the capacity of the cut $(S \rightarrow T)$ only).

Proof:

$$\begin{aligned}
\text{val}(f) &= \sum_{a \in \delta^+(s)} f(a) - \sum_{a \in \delta^-(s)} f(a) + \sum_{v \in S \setminus \{s\}} \left(\sum_{a \in \delta^+(v)} f(a) - \sum_{a \in \delta^-(v)} f(a) \right) \\
&= \sum_{a \in \delta^+(S)} f(a) - \sum_{a \in \delta^-(S)} f(a) = \sum_{a \in \delta(S, T)} f(a) - \sum_{a \in \delta(T, S)} f(a) \\
&= f(S, T) \leq \sum_{a \in \delta(S, T)} u(a) - \sum_{a \in \delta(T, S)} 0 \\
&= u(S)
\end{aligned} \tag{12}$$

■

3.a.b Residual graph & augmenting paths

For every arc $a = (i, j) \in A$, we define a reversed arc $\tilde{a} = (j, i)$ and the residual capacities (assuming that $\tilde{a} \notin A$):

$$u_f(a) = u(a) - f(a) \quad \text{and} \quad u_f(\tilde{a}) = f(a) \tag{13}$$

In the general case where both directions of an edge can be present, we must use

$$u_f(a) = u(a) - f(a) + f(\tilde{a}) \tag{14}$$

The residual graph is the capacitated graph $D_f = (V, A_f, u_f)$ with

$$A_f = \{a \in A \cup \tilde{A} : u_f(a) > 0\} \tag{15}$$

An f -augmenting path is an s - t path in the residual graph D_f .

To augment f by γ along an f -augmenting path P means performing, for every $a \in P$:

- $f(a) \leftarrow f(a) + \gamma$ if $a \in A$
- $f(a) \leftarrow f(a) - \gamma$ if $a \in \tilde{A}$

3.a.c Strong duality

Theorem (Optimality criterion): An s - t flow f is maximal if and only if there is no f -augmenting path.

Proof:

1. If there is an augmenting path P , the flow can be augmented along this path (by the minimum residual capacity $\min\{u_f(a) : a \in P\}$).
2. If no such path exists, then s and t are separated in the residual graph. Let S denote the connected component of D_f to which s belongs, and $T = V \setminus S$. The residual capacity is $u_f(S, T) = 0$, which means that the net flow $f(S, T)$ is equal to the cut capacity $u(S)$ (if an edge across the cut is not properly saturated, this contradicts the disconnectedness of S and T). The previous proposition implies that the flow f is maximal and the cut (S, T) is minimal in D .



Theorem (Max flow / min cut): The maximum value of an s - t flow is equal to the minimum value of an s - t cut.

3.b Ford-Fulkerson

3.b.a Pseudocode

Input: a digraph $D = (V, A)$ with capacities u , two vertices s and t

1. Set $f(a) = 0$ for all $a \in A$
2. While there is an f -augmenting path:
 1. Select an f -augmenting path P
 2. Augment f along P by $\min_{a \in P} u_f(a)$

Output: a maximum s - t flow f

Questions:

- How do we find / select an augmenting path?
- Does the algorithm terminate, and if so when?

3.b.b Complexity

Each iteration of the Ford-Fulkerson loop takes $O(|A|)$ time

If the capacities u are integral, so are the flow augmentations.

Theorem: If the capacities u are integral, the Ford-Fulkerson algorithm returns an integral maximum s - t flow in $O(|A| \times \text{val}_{\max})$ time, where $\text{val}_{\max}$ is the maximum value of an s - t flow.

3.c Edmonds-Karp

3.c.a Pseudocode

Input: a digraph $D = (V, A)$ with capacities u , two vertices s and t

1. Set $f(a) = 0$ for all $a \in A$
2. While there is an f -augmenting path:
 1. Select an f -augmenting path P *with minimum number of edges*
 2. Augment f along P by $\min_{a \in P} u_f(a)$

Output: a maximum s - t flow f

Questions:

- How do we select such an augmenting path?
- Why does it improve the complexity?

3.c.b Complexity

The Edmonds-Karp loop is crossed at most $|A| \times |V|$ times.

Proof: We can show that

- The (unweighted) distance $\text{dist}_{D_f}(s, t)$ in the residual graph is nonincreasing
- It can only remain constant for at most $|A|$ iterations

Theorem: The Edmonds-Karp algorithm returns a maximum s - t flow in $O(|A|^2 \times |V|)$ time.

3.d Minimum mean cycle-canceling (for minimum cost b -flows)

Pseudocode

Input: a digraph $D = (V, A)$ with capacities $\ell \leq u$, costs c and inflows b

1. Find an initial b -flow f
2. While there is an f -augmenting cycle with negative cost
 1. Select an f -augmenting cycle C with minimum mean cost
 2. Augment f along C by $\min_{a \in C} u_f(a)$

Output: a minimum cost b -flow f

Questions:

- How do we find an initial b -flow?
- How do we select an f -augmenting cycle with minimum mean cost?

Complexity

An initial b -flow can be found in $O(|A| \times |V|)$ time (see Ex 6.4).

An f -augmenting cycle of minimum mean cost can be found in $O(|A| \times |V|)$ time (see Ex 6.3).

Theorem 6.10: The cycle-canceling algorithm returns a minimum cost b -flow in $O(|A|^3 |V|^2 \log |V|)$ time.

4 Linear programming for flows

4.a Formulation

We can formulate the maximum flow problem as follows:

$$\begin{aligned}
 \max_x \quad & \sum_{a \in \delta^+(s)} x_a - \sum_{a \in \delta^-(s)} x_a \\
 \text{s.t.} \quad & \sum_{a \in \delta^-(v)} x_a = \sum_{a \in \delta^+(v)} x_a \quad \forall v \in V \setminus \{s, t\} \\
 & 0 \leq x_a \leq u(a) \quad \forall a \in A
 \end{aligned} \tag{16}$$

We will prove the following results later in the course:

- The constraint matrix of the max flow LP is “totally unimodular”.
- The minimum cut is the “Lagrangian dual” of the maximum flow, up to integrality.

Exercise: prove this

The equivalent max-flow formulation

$$\begin{aligned}
& \max_{x \geq 0, q} && q \\
& \text{s.t.} && \sum_{a \in \delta^+(s)} x_a - \sum_{a \in \delta^-(s)} x_a = q \\
& && \sum_{a \in \delta^+(t)} x_a - \sum_{a \in \delta^-(t)} x_a = -q \\
& && \sum_{a \in \delta^-(v)} x_a = \sum_{a \in \delta^+(v)} x_a \quad \forall v \in V \setminus \{s, t\} \\
& && x \leq u
\end{aligned} \tag{17}$$

has the following dual

$$\begin{aligned}
& \min_{y, z} && \sum_a u_a z_a \\
& \text{s.t.} && z_a \geq y_v - y_w \quad \forall a = (v, w) \in A \\
& && y_s - y_t \geq 1 \\
& && z_a \geq 0 \quad \forall a \in A
\end{aligned} \tag{18}$$

where $z_a = 1$ if we select an edge in the cut and $y_v = 1$ if $v \in S$. Using binary variables, the first inequality forces a to be in the cut if v and w are in separate connected components. ■

4.b Polyhedral interpretation

A general result in polyhedral geometry (the Minkowski-Weyl theorem) states that every polyhedron P can be written as

$$P = \left\{ \sum_i \lambda_i x_i + \sum_j \mu_j y_j : \lambda \geq 0, \mu \geq 0, \sum_i \lambda_i = 1 \right\} \tag{19}$$

The flow version of this result is Proposition 6.13: every s - t flow can be decomposed as a positive sum of flows along elementary s - t paths or elementary cycles.

4.c Homework

Exercise 6.12: taxi fleet. A fleet of taxis has p clients to serve on a given day, known in advance. For each client i , we know their origin o_i , destination d_i , departure time h_i and trip duration t_i . How to compute the minimum number of taxis necessary to satisfy the demand?

Exercise 6.14: consistent rounding. Let $A = (a_{ij}) \in \mathbb{R}^{m \times n}$ be a matrix. We want to round each entry either up or down to an integer, such that the rounding of each row (resp. column) sum equals the sum of the roundings. Show that this is always possible.